

20251223 Spring+Mybatis复习答案

1、请说一下对 MyBatis 的缓存的理解

MyBatis 的缓存分为一级缓存和二级缓存，核心是减少数据库交互、提升查询性能：

- 一级缓存：也叫本地缓存，默认开启，作用域为 SqlSession（会话级别）。同一个 SqlSession 内执行相同的 SQL 查询（参数、SQL 语句等完全一致），MyBatis 会优先从一级缓存取数据；执行增删改、提交事务、关闭 SqlSession 等操作时，一级缓存会清空，保证数据一致性。
- 二级缓存：也叫全局缓存，作用域为 Mapper（命名空间级别），默认关闭需手动开启。开启后不同 SqlSession 访问同一 Mapper 的相同查询，可共享二级缓存数据；实体类需实现 Serializable 接口支持序列化，增删改操作会清空对应 Mapper 的二级缓存。
- 缓存执行顺序：查询时先查一级缓存→一级缓存无则查二级缓存→二级缓存无
- 则查数据库→结果回写至一级、二级缓存（二级缓存开启时）。

2、如果要让你实现批量删除你会怎么操作

MyBatis 实现批量删除主要有三种常见方式，适配不同业务场景：

1. 基于 IN 子句实现：Mapper 接口接收集合参数，在 XML 中通过 foreach 标签遍历集合生成 IN 条件，适用于中小批量数据删除；需注意 IN 子句的参数数量限制（数据库默认约 1000 个），避免超出限制导致 SQL 报错。
 1. 分批删除：针对大数据量删除场景，将大集合拆分为多个小批次（如每批 500 条），循环调用单批删除方法，避免 IN 子句过长，同时需控制事务保证批量操作的原子性。
2. 单步执行SQL语句，最后统一进行提交

3、什么是 MyBatis 中延迟加载，如何开启延迟加载（全局、局部）

（1）延迟加载定义

延迟加载（懒加载）指 MyBatis 查询关联对象（一对一、一对多关联）时，不会立即查询关联数据，仅在真正调用关联对象的属性（如 getXXX () 方法）时，才执行关联查询；核心目的是减少不必要的数据库交互，提升查询性能，与立即加载（查询主对象时同步查关联对象）相对。

（2）开启方式

- 全局开启：在 MyBatis 核心配置文件 mybatis-config.xml 中，通过 settings 标签配置：开启 lazyLoadingEnabled（全局延迟加载开关），关闭 aggressiveLazyLoading（仅调用关联属性时加载），可按需配置 lazyLoadTriggerMethods 指定触发延迟加载的方法。
- 局部开启 / 关闭：在 Mapper.xml 的关联查询标签（association/collection）中，通过 fetchType 属性覆盖全局配置，fetchType="lazy" 表示局部开启延迟加载，fetchType="eager" 表示局部关闭（立即加载）。

4、请说一下 \${} 与 #{} 的区别与联系

(1) 联系

两者均为 MyBatis 中接收 Java 参数的占位符，用于替换 SQL 中的参数位置，是 Mapper.xml/ 注解中传递参数的核心方式。

(2) 区别

- 底层实现：#{ } 基于 PreparedStatement 预编译，\${ } 基于 Statement 字符串拼接；
- 注入风险：#{ } 自动对参数转义，能防止 SQL 注入，\${ } 直接拼接参数，存在注入风险；
- 参数处理：#{ } 会为参数添加引号，\${ } 直接拼接无引号；
- 适用场景：#{ } 用于传递普通字段值（如 ID、名称），\${ } 仅用于传递 SQL 片段（如表名、列名、排序字段）；

5、你在项目中 MyBatis 传参你用过哪些方案（至少例举 3 种，说明每一种的特点）

1. 单个基本类型参数：参数名可任意（MyBatis 3.4 + 自动识别），也可通过 @Param 指定名称，XML 中直接用 #{参数名} 接收；特点是简单易用，但仅支持单个参数，多参数场景不适用。
2. @Param 注解传参：为每个参数指定唯一名称，XML 中通过该名称接收；特点是直观，适配 2-3 个参数的场景，参数过多时代码冗余。
3. 实体类传参：将参数封装为实体类对象，XML 中通过 #{实体属性名} 接收；特点是适配参数与实体字段匹配的场景，结构清晰，便于扩展。
4. Map 传参：将参数以键值对存入 Map，XML 中通过 #{key} 接收；特点是灵活，无需定义实体类，适配临时拼接参数的场景，但无类型校验，可读性差。

6、你之前团队中关于 SQL 编写有什么规范【例举 5 条】

1. 命名规范：Mapper 接口方法名采用“动词 + 名词”驼峰命名，XML 中 SQL 标签 ID 与接口方法名一致；表 / 列别名简短且语义化，避免无意义别名。
2. 性能规范：禁止 SELECT *，需显式指定查询列；IN 子句参数数量不超过 1000，大数据量分批处理；关联查询层级不超过 3 层，避免笛卡尔积；索引字段禁止做函数运算。
3. 安全规范：优先使用 #{ } 占位符，仅在表名 / 列名等必须拼接的场景使用 \${ }，且需做参数白名单校验，防止 SQL 注入。
4. 格式规范：SQL 关键字独占一行，子句缩进；复杂 SQL 添加业务说明注释；避免冗余空格和空行，保持 XML 整洁。
5. 容错规范：WHERE 条件中对字符串参数做空值判断；列表查询必加分页 LIMIT；参数类型与数据库字段类型保持一致，避免隐式类型转换。

7、最佳实践中，通常一个 Xml 映射文件，都会写一个 Dao 接口与之对应，请问，这个 Dao 接口的工作原理是什么？Dao 接口里的方法，参数不同时，方法能重载吗？

(1) Dao 接口工作原理

MyBatis 通过 JDK 动态代理实现 Dao 接口：启动时扫描 Dao 接口并生成动态代理类

（MapperProxy）；调用接口方法时，实际执行代理类的 invoke 方法，代理类根据方法名、参数匹配 XML 中对应 ID 的 SQL 标签；MyBatis 拼接 SQL、处理参数、执行数据库操作并返回结果，接口无需实现类，核心是“接口全限定名 + 方法名”与 XML SQL 的映射关系。

(2) Dao 接口方法能否重载

Dao 接口方法禁止重载，重载后 MyBatis 无法正确匹配 SQL：MyBatis 通过“接口全限定名 + 方法名”匹配 SQL，而非参数；重载方法名相同，会导致 MyBatis 无法区分对应的 XML 标签，抛出绑定异常，需通过不同方法名区分（如 getUserByName、getUserByAge）。

8、了解单例模式吗？能不能写一个实现代码

(1) 单例模式定义

单例模式是创建型设计模式，核心是保证一个类在程序运行期间仅有一个实例，并提供全局访问点；适用于工具类、连接池、配置类等场景，避免重复创建对象浪费资源。

```
public class SingletonLazy {  
    // 1. 私有静态实例，volatile防止指令重排  
    private static volatile SingletonLazy INSTANCE;  
  
    // 2. 私有构造方法  
    private SingletonLazy() {}  
  
    // 3. 双重校验锁获取实例  
    public static SingletonLazy getInstance() {  
        // 第一层校验：避免已创建实例后，每次调用都加锁  
        if (INSTANCE == null) {  
            synchronized (SingletonLazy.class) {  
                // 第二层校验：防止多线程同时进入第一层后，重复创建实例  
                if (INSTANCE == null) {  
                    INSTANCE = new SingletonLazy();  
                }  
            }  
        }  
        return INSTANCE;  
    }  
}
```

9、如何排查慢查询

慢查询排查核心流程为“定位→分析→优化→验证”：

1. 开启慢查询日志：MySQL 中通过配置开启慢查询日志，设置执行时间阈值（如 0.5 秒），记录超过阈值的 SQL 及未使用索引的 SQL。
2. 定位慢查询 SQL：查看慢查询日志，或用 mysqldumpslow 工具按执行时间排序分析；通过 show processlist 查看当前运行的慢 SQL。
3. 分析执行计划：执行 EXPLAIN + 慢查询 SQL，重点关注 type（访问类型，避免全表扫描）、key（实际使用的索引）、rows（预估扫描行数）、Extra（文件排序 / 临时表等需优化项）。
4. 优化慢查询：为关键字段添加索引（覆盖索引优先）；优化 SQL（减少 JOIN、拆分大 SQL、批量操作）；调整 MySQL 缓存参数；数据量大时考虑分库分表 / 读写分离。
5. 验证优化效果：重新执行 EXPLAIN 确认指标优化，监控 SQL 执行时间对比优化前后效果。

10、什么是类加载，简述类加载的过程

(1) 类加载定义

类加载是 JVM 通过类加载器将磁盘上的.class 字节码文件加载到内存，经验证、解析、初始化后，生成可被 JVM 使用的 Class 对象的过程；核心遵循双亲委派模型，保证类加载的唯一性。

(2) 类加载过程

1. 加载：类加载器根据类的全限定名读取.class 文件字节流，转换为 Class 对象，通过双亲委派模型选择类加载器，避免类重复加载。
2. 验证：校验.class 文件的合法性（格式、语义、安全性），防止恶意字节码攻击。
3. 准备：为类的静态变量分配内存，设置默认初始值（如 int 默认 0），不执行赋值语句。
4. 解析：将类的符号引用（如类名、方法名字符串）转换为直接引用（内存地址、指针）。
5. 初始化：执行类的静态代码块和静态变量赋值语句，是类加载的最后一步；触发时机包括首次创建实例、调用静态方法 / 变量、反射调用、初始化子类（先初始化父类），至此类真正可用。